ADR-001: Separar el backend en capas

Decisión de arquitectura de software

Estado: Implementada en el módulo MVP. Los cinco archivos de inventario y préstamos quedaron organizados por capas y, en la entrega final, sus controladores pasaron a ser la API REST del ADR-005. Los diez archivos restantes, relacionados con el dashboard, usuarios y otras funciones, no se migraron y quedan como trabajo futuro.

1. Problema que resuelve (semana 3)

En la semana 3 identificamos que los módulos de Usuarios, Inventario, Préstamos y Devoluciones estaban muy relacionados dentro del mismo backend. Esta forma de trabajar podía hacer que el código fuera cada vez más difícil de mantener, probar y modificar.

Al revisar el código en la semana 8, encontramos un ejemplo concreto: el archivo prestamos.php se encargaba de conectarse a la base de datos, construir una consulta que relaciona tres tablas y devolver la respuesta en formato JSON. Algo parecido ocurría en los otros catorce archivos PHP. Esto también generaba repetición, porque si el Dashboard necesitaba la misma información, era necesario volver a escribir una consulta similar.

2. Cambio propuesto

La decisión es organizar el backend en capas, de manera que cada una tenga una responsabilidad clara. Esta estructura corresponde a la que se muestra en el diagrama de componentes (docs/Diagramas/C4 #3.pdf).

Capa	Responsabilidad	Ubicación
Controladores	Reciben las solicitudes de la página y devuelven las respuestas. No ejecutan consultas SQL.	src/Inventario/Http/ y src/Prestamos/Http/ (en la semana 13 eran src/objetos.php, src/prestamos.php y otros)
Servicios	Coordinan las operaciones del sistema, como registrar un préstamo o cambiar el estado de un equipo.	src/Inventario/Servicio/ y src/Prestamos/Servicio/
Reglas de dominio	Definen qué operaciones están permitidas, por ejemplo, los cambios válidos de estado de un equipo.	src/Inventario/Dominio/ EstadoObjeto.php
Repositorios de datos	Centralizan las consultas SQL para evitar que se repitan en distintos archivos.	src/Inventario/Repositorio/ y src/Prestamos/Repositorio/

Los elementos que comparten varios módulos —como Conexión, Transacción, Contenedor y RespuestaJson— se mantienen en src/Compartido/ y src/Config/.

En la semana 13 los archivos que usa la página conservaron sus nombres dentro de la carpeta src/, y en index.html solo se cambió la ruta de las llamadas (antes api/, ahora src/). En la entrega final esos controladores se convirtieron en la API REST descrita en el ADR-005, sin cambiar los servicios ni los repositorios.

3. Código afectado

Antes	Después
src/prestamos.php reunía la conexión, la consulta y la respuesta JSON en un solo archivo.	El archivo solicita los datos a PrestamoServicio y la consulta queda en PrestamoRepositorio.
src/objetos.php y src/objetos_disponibles.php concentraban tareas similares.	Solicitan la información a InventarioServicio y las consultas quedan en ObjetoRepositorio.
src/registrar_prestamo.php y src/devolver.php llamaban a procedimientos almacenados.	Ahora llaman a PrestamoServicio (véanse también ADR-003 y ADR-004). En la entrega final quedaron en PrestamoControlador (ADR-005).
src/db.php repetía la conexión utilizada por varios archivos.	La nueva estructura utiliza src/Config/Conexion.php. db.php se conserva mientras se migran los archivos pendientes.

4. Atributo de calidad que mejora (semana 8)

El principal atributo que mejora es la mantenibilidad. En la semana 8 observamos que no existía un único lugar para cambiar la forma de consultar un equipo o un préstamo. Con la nueva organización, esas consultas quedan centralizadas en el repositorio correspondiente.

Además, al separar las reglas de negocio del acceso a la base de datos, cada parte se puede revisar y modificar sin tocar las demás.

Otras opciones que descartamos

- Usar un framework como Laravel: se descartó porque implicaría aprender una herramienta nueva y reescribir una parte importante del sistema. El problema identificado era de organización del código, no de falta de un framework.
- Mantener las reglas dentro de procedimientos almacenados: se descartó porque esa era precisamente una de las dificultades detectadas; esas reglas resultaban más difíciles de probar y modificar.

Ventajas y costos

- Ventaja: cada responsabilidad queda ubicada en un lugar definido, lo que facilita comprender el código y probar sus partes por separado.
- Costo: aumenta la cantidad de archivos y, mientras termina la migración, convivirán temporalmente la estructura anterior y la nueva.

Cómo lo comprobamos

- Con la base creada con BDSQL V3.sql, usar la página para ver el inventario, registrar un préstamo y hacer su devolución. Todo debe funcionar igual que antes del cambio.
- Verificar que los controladores ya migrados no contengan consultas SQL. Los archivos que aún no se han migrado pueden conservarlas temporalmente.